



بنك تونس والإمارات
Banque de Tunisie et des Emirats

République Tunisienne
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
Université de Tunis El Manar
Institut Supérieur d'Informatique

INSTITUT
SUPERIEUR
INFORMATIQUE
المعهد العالي للإعلامية



Soutenance de Stage de Fin d'Études

En vue de l'obtention du Diplôme National de Mastère Professionnel

Spécialité : Sécurité des Systèmes d'Informations et des Infrastructures

BTE Security AI Agent

Agent IA pour la revue automatisée de code

Encadrants

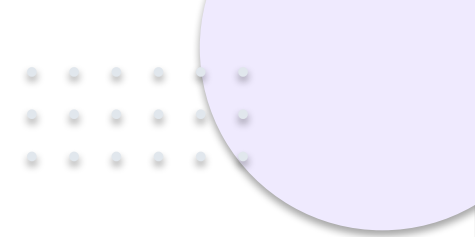
M. Kamel KAOUECH — professionnel

Mme Ghayet El Mouna ZHIOUA — académique

Réalisé par

Ghaith FERCHICHI

Sommaire



1

Contexte général

Organisme d'accueil, étude de l'existant, problématique et méthodologie

2

État de l'art et choix technologiques

Concepts DevSecOps, spécification des besoins et technologies retenues

3

Conception

Architecture de l'agent — « un cerveau, et non un simple pipeline »

4

Réalisation et résultats

Sécurité, démonstration, observabilité et validation

5

Conclusion et perspectives

Limites, perspectives et bilan du projet



INTRODUCTION

Une seule injection SQL fusionnée dans une API de paiement
peut compromettre l'intégrité du système d'information de la banque.

À la BTE, la revue de sécurité du code reste **entièrement manuelle**.
Aujourd'hui, aucun contrôle automatique n'arrête une faille.

01

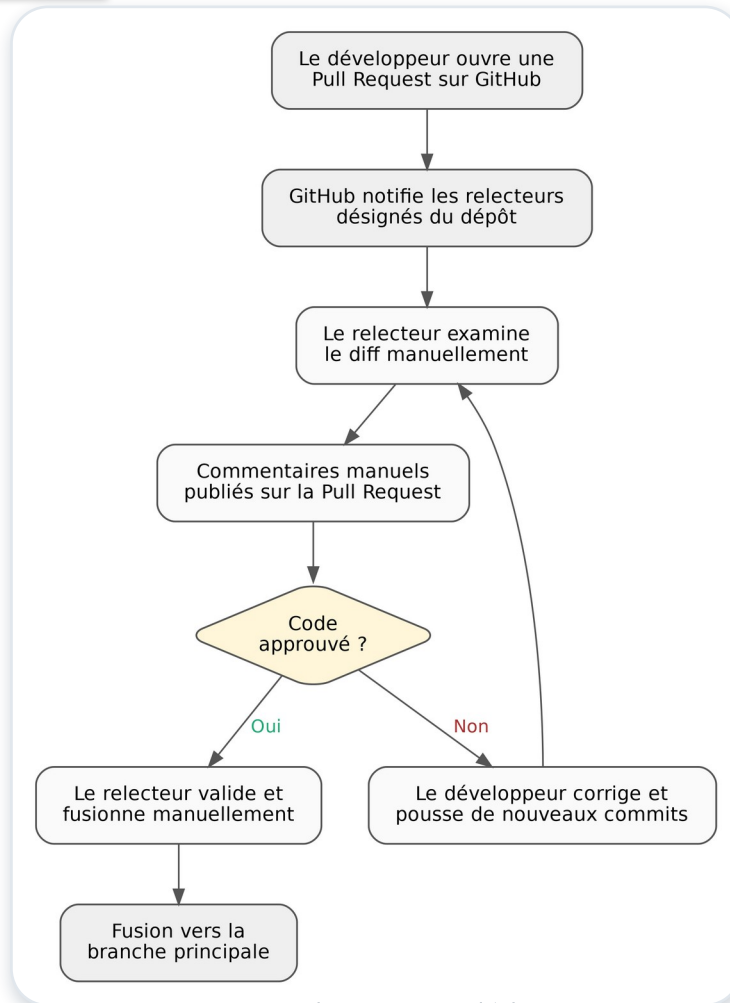
Contexte



Organisme d'accueil, étude de l'existant et méthodologie de travail



Organisme d'accueil et étude de l'existant



Processus de revue actuel à la BTE

Banque de Tunisie et des Émirats

- **Banque universelle** — particuliers, professionnels, entreprises
- **DCIO — sécurité opérationnelle** — cadre d'accueil du stage
- **Convention État tunisien · ADIA** — création en 1982, vocation élargie

Une revue intégralement manuelle

aucune analyse statique ni gate CI/CD n'est greffé sur le cycle de livraison.

Problématique



Délai pouvant atteindre 24 h

le temps qu'une Pull Request soit revue à la main



Absence de gate CI/CD

une Pull Request vulnérable peut être fusionnée sans blocage



Qualité de revue variable

selon la disponibilité et l'expertise du relecteur



Angles morts

secrets commités et dépendances vulnérables non détectés à l'œil nu

Objectifs et contribution

Un **agent IA autonome** qui revoit chaque Pull Request selon l'**OWASP Top 10** en **quinze minutes**, publie les correctifs sur GitHub, et s'exécute **intégralement en local** — aucun code ne quitte la banque.

24 h → 15 min

délai de revue

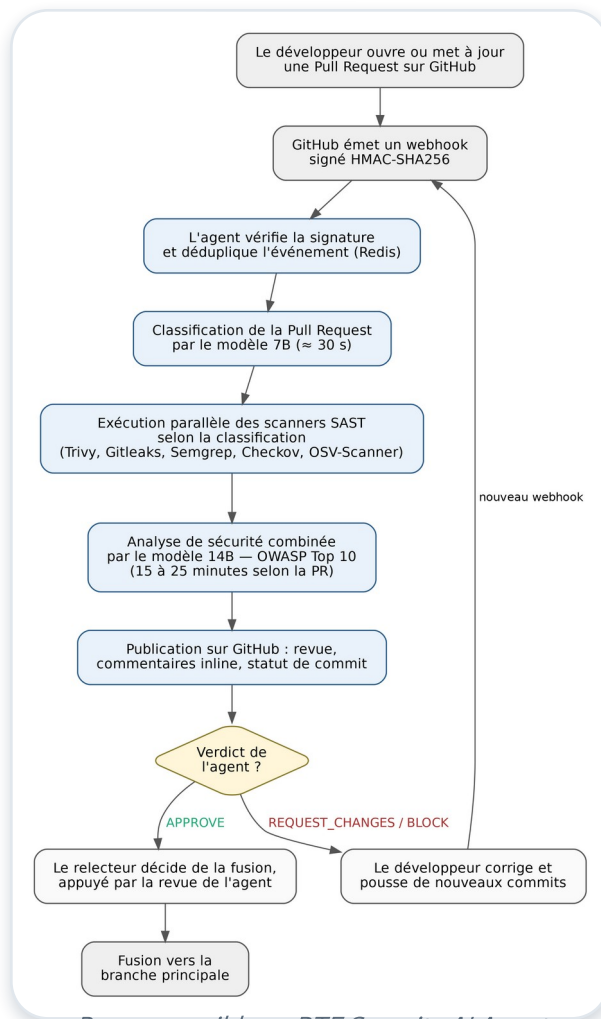
7 / 7

vulnérabilités détectées

0

ligne de code ne sort du VPS

Solution proposée



Processus cible — BTE Security AI Agent

Un agent qui prend en charge tout le cycle

- » Webhook signé, pipeline automatique
- » Cinq scanners SAST et deux LLM locaux
- » Revue OWASP, score et verdict
- » Gate CI/CD (APPROVE / CHANGES / BLOCK)
- » Inférence locale — conformité BCT

Délai : 15-25 min vs jusqu'à 24 h en manuel

Méthodologie de travail

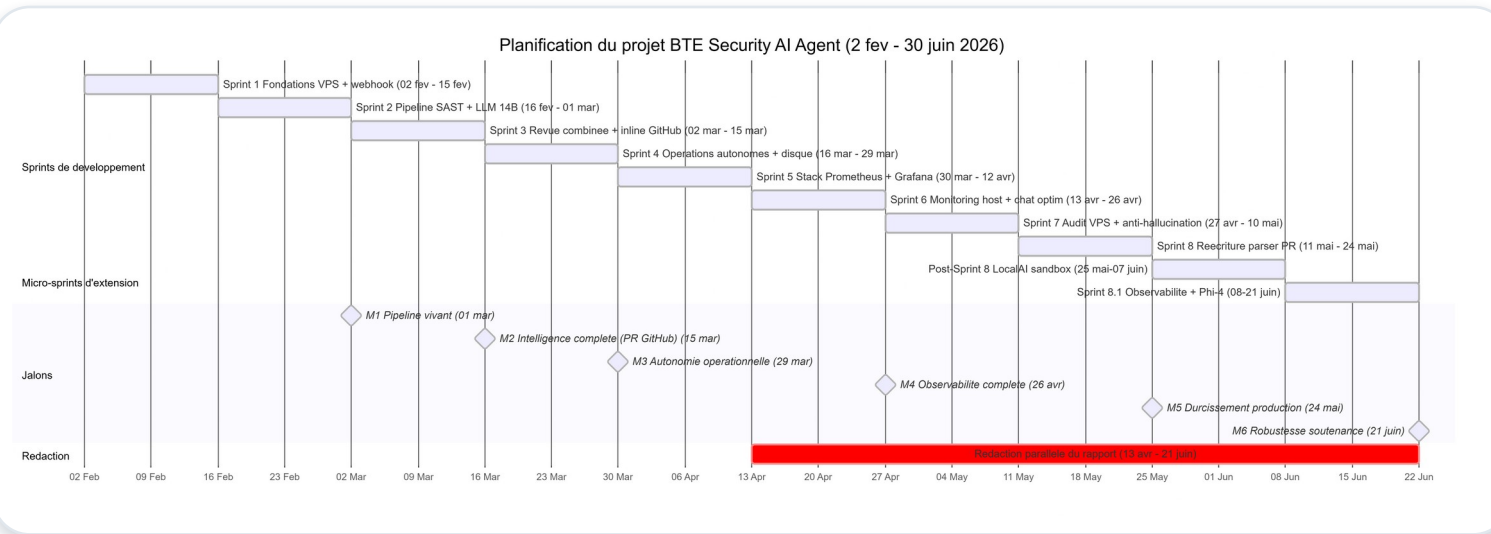


Diagramme de Gantt — dix sprints sur cinq mois

Méthode Scrumban

- **Scrum + Kanban**
cadence de sprints, flux à WIP = 1
- **Dix sprints sur cinq mois**
de la fondation Docker à l'observabilité
- **Système « pull »**
chaque incident de production devient la carte prioritaire

02

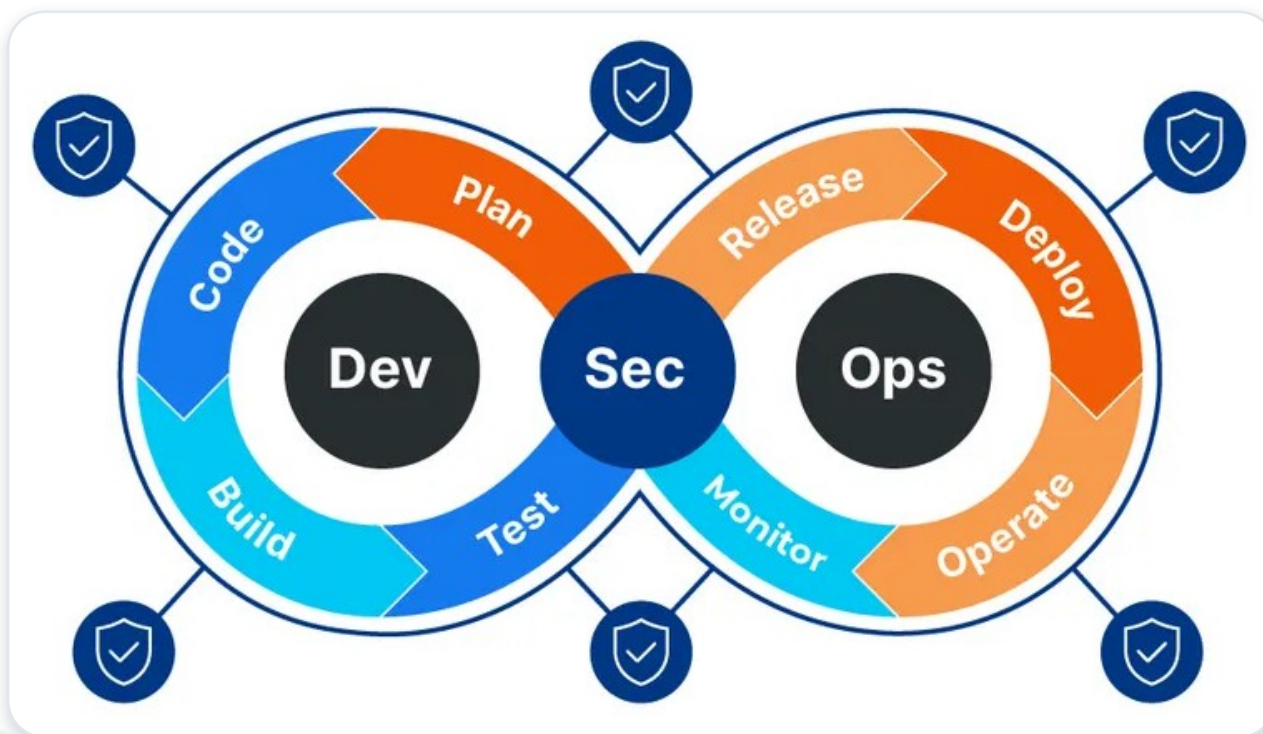
État de l'art



Concepts DevSecOps, spécification des besoins et choix technologiques



DevSecOps et approche shift-left

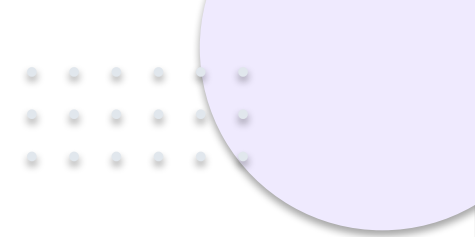


Cycle de vie DevSecOps

Intégrer la sécurité au plus tôt

- **Security as Code**
politiques de sécurité versionnées et automatisées
- **Shift Left**
détecter la vulnérabilité dès la Pull Request
- **Le verdict comme gate**
un contrôle technique conditionne la fusion

Spécification des besoins



Besoins fonctionnels

- Détecter les vulnérabilités (5 scanners)
- Produire la revue OWASP (score, verdict, inline)
- Publier sur GitHub + gate CI/CD
- Persister chaque revue (PostgreSQL)
- Chat opérationnel temps réel (19 outils)
- Sécuriser la plateforme (HMAC, secrets exclus)

Besoins non fonctionnels

- Confidentialité — 100 % local (conformité BCT)
- Performance — 15 à 25 min par revue
- Résilience — circuit breaker et fallbacks
- Fiabilité — reprise après redémarrage du conteneur
- Observabilité — Prometheus / Grafana

Choix des modèles LLM — benchmark sur le matériel cible

Modèle	Taille	Vitesse	Précision outils	Décision
qwen2.5-coder:7b	4,7 Go	~5 tok/s	80 %	Classification + chat
qwen2.5-coder:14b	9,0 Go	~3,2 tok/s	80 %	Analyse de sécurité
llama3.2:3b	2,0 Go	~8 tok/s	0 %	Écarté
granite3.1-dense:2b	1,6 Go	~8,5 tok/s	0 %	Écarté

Architecture à deux modèles : le 7B classifie en ~30 s, le 14B mène l'analyse approfondie — à précision d'appel d'outils égale (80 %).

Ollama : +22 %
de débit vs LocalAI, sur GGUF identique (benchmark croisé).

Technologies retenues

5

Cinq scanners SAST

Trivy · Gitleaks · Semgrep · Checkov · OSV

IA

Deux LLM locaux

qwen2.5-coder 7B (classer) + 14B (analyser)



Orchestration

LangGraph — graphe d'état, reprise, routage

Sortie SAST nettoyée (-52 % de tokens) avant transmission au LLM.

Positionnement par rapport aux solutions existantes

Solution	Données / inférence	LLM	100 % local
Snyk	Code envoyé au cloud	Limité	Non
SonarQube	Auto-hébergeable	Non	Règles statiques
Copilot Autofix	Cloud	Oui	Non
BTE Security AI Agent	Rien ne sort du VPS	Oui	OUI

Différenciateur : l'inférence 100 % locale garantit la confidentialité bancaire et la conformité BCT.

03

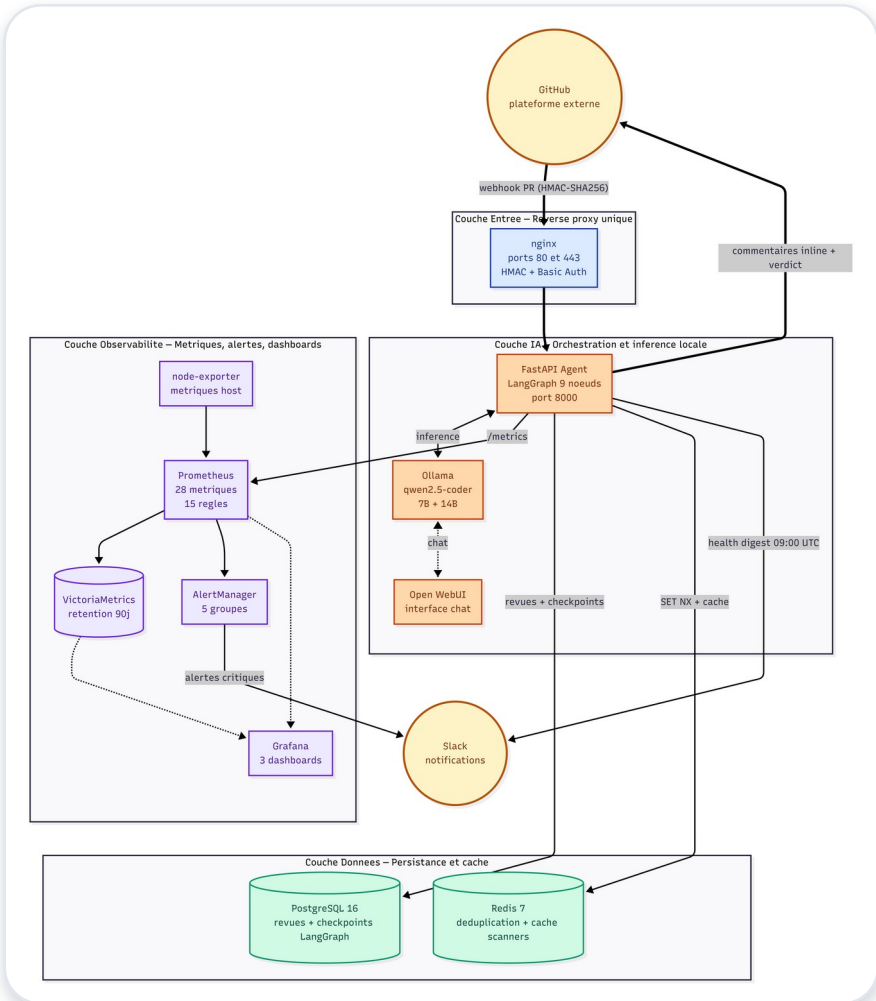
Conception



Architecture de l'agent — « un cerveau, et non un simple pipeline »



Architecture globale de la solution



Onze conteneurs répartis en quatre couches

Couche entrée

nginx — webhook HMAC

Couche IA

FastAPI + LangGraph + Ollama

Couche données

PostgreSQL + Redis

Observabilité

Prometheus · Grafana

Un agent autonome, et non un simple appel LLM

1 Perception

événements webhook GitHub

2 Raisonnement

classe la PR, route le graphe d'état

3 Action

commente, fixe le gate, escalade

4 État et reprise

checkpoints LangGraph (PostgreSQL)

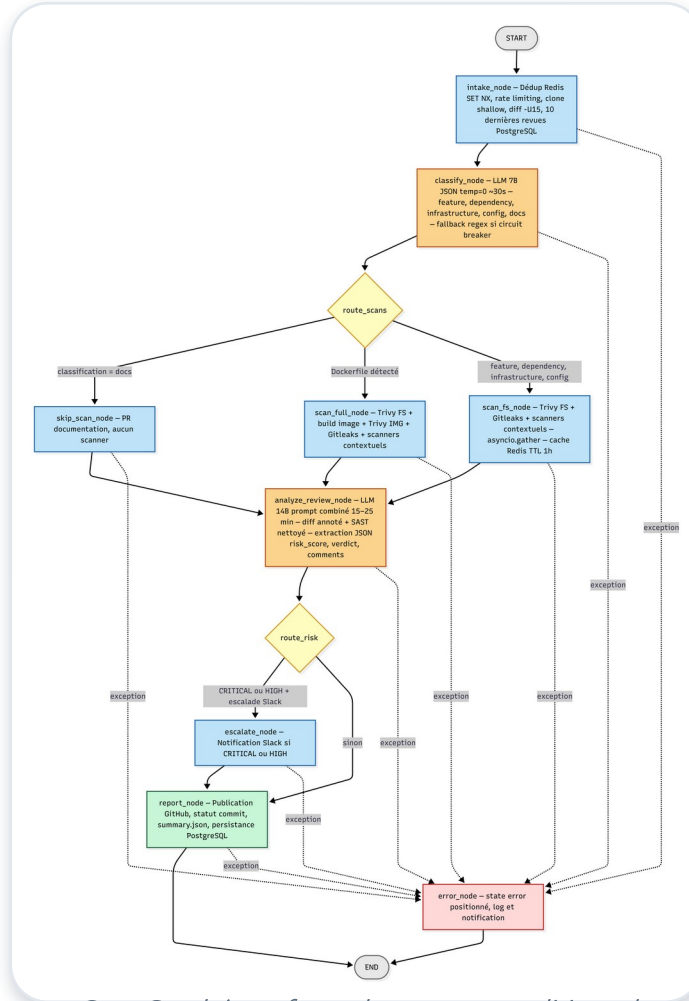
5 Usage d'outils

dix-neuf outils réels (Docker, Prometheus...)

6 Auto-exploitation

scheduler, health digest, gardien disque

Le pipeline de revue — graphe d'état LangGraph



StateGraph à neuf nœuds, routage conditionnel

1 webhook — HMAC + déduplication

2 classify — LLM 7B (~30 s)

3 route — scanners selon la PR

4 scan — cinq SAST en parallèle

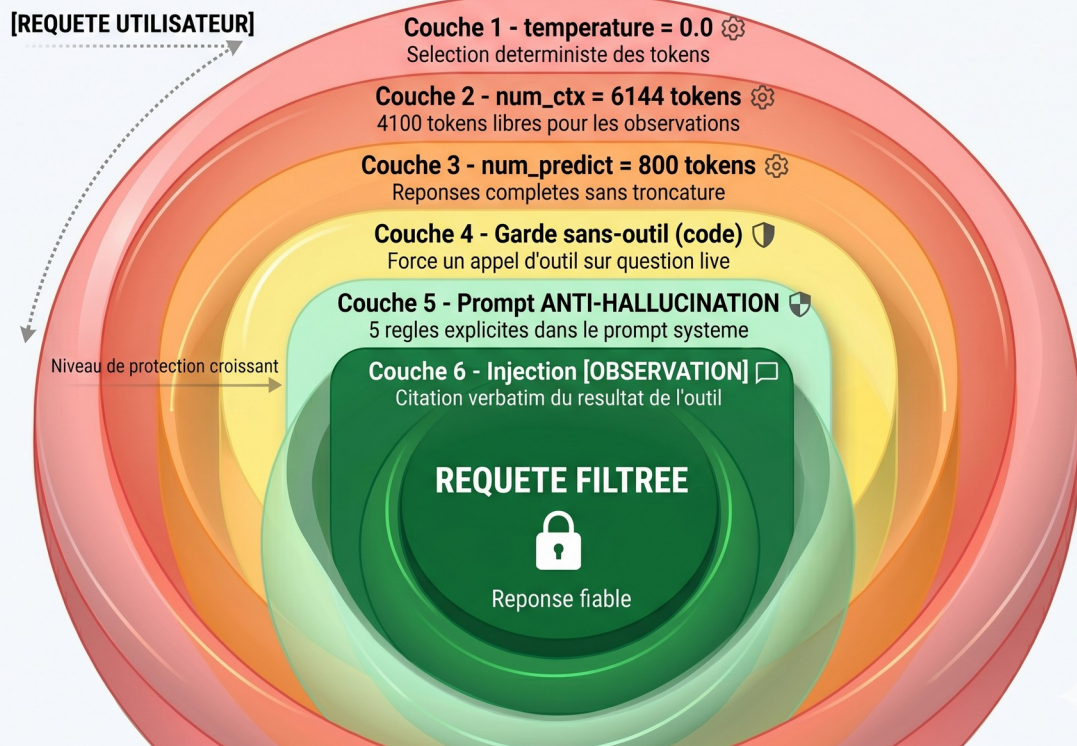
5 analyze — LLM 14B + OWASP

6 verdict — APPROVE/CHANGES/BLOCK

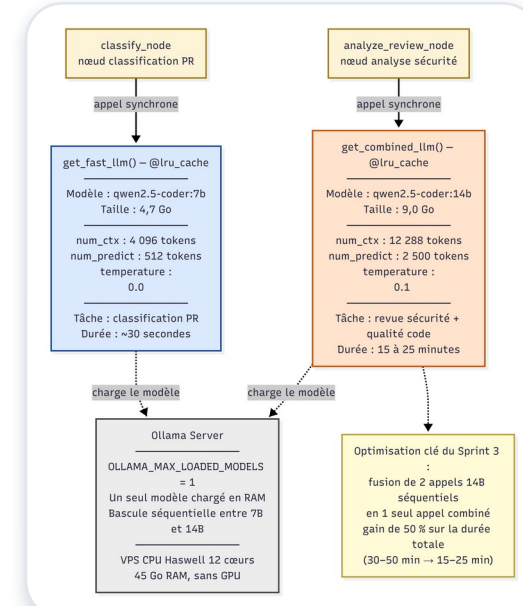
7 publish — commentaires + gate

Fiabilité — anti-hallucination et deux modèles

Systeme anti-hallucination - 6 couches de protection empilees



Six couches de protection



Deux modèles : 7B pour classer, 14B pour analyser

Garantir la fiabilité des sorties du LLM

temperature=0 · num_ctx maîtrisé · garde sans-outil · prompt anti-hallucination · parser éliminant les lignes inexistantes · fusion réduisant le temps de moitié.

04

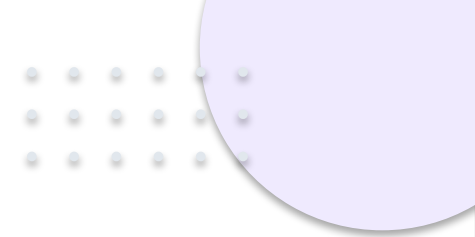
Réalisation



Environnement, sécurité en action, démonstration et validation



Environnement de travail



Environnement matériel — VPS de production

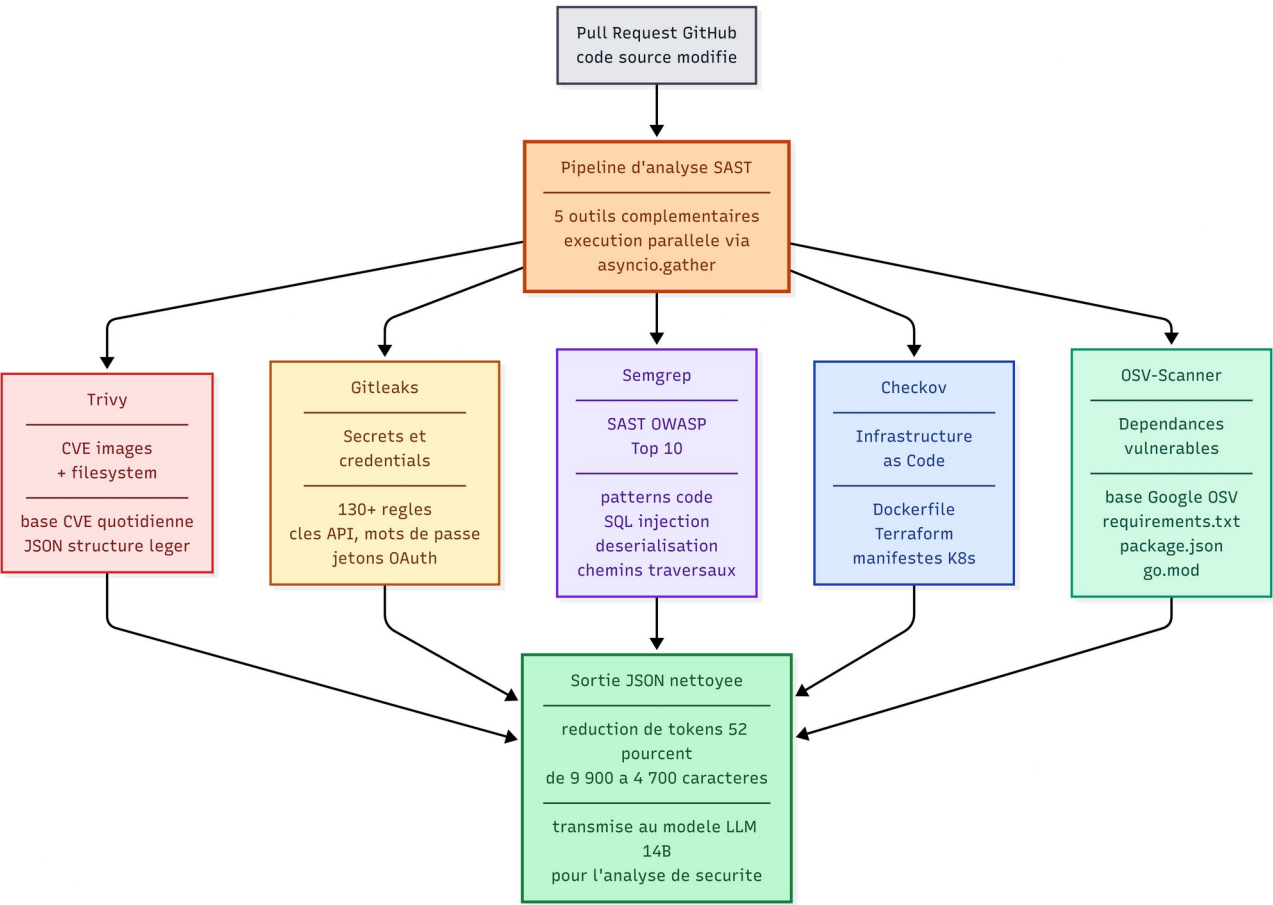
- CPU : 12 cœurs Intel Haswell (AVX2)
- RAM : 45 Go · Disque : 290 Go
- **GPU : aucun — inférence CPU uniquement**
- OS : Ubuntu Linux

Environnement logiciel — pile open source

- Docker 29.4 — onze conteneurs
- Python 3.12 · FastAPI · LangGraph 1.1+
- PostgreSQL 16 · Redis 7 · nginx
- Trivy · Gitleaks · Semgrep · Checkov · OSV-Scanner

Une contrainte assumée : démontrer la faisabilité sur une infrastructure bancaire réelle, sans accélérateur.

Défense en profondeur — cinq scanners et un LLM



Cinq outils SAST en parallèle — OWASP Top 10 2025

Trivy

CVE et images

Gitleaks

secrets

Semgrep

OWASP code

Checkov

IaC

OSV-Scanner

dépendances

Complémentarité

Chaque scanner couvre une classe de risque — secrets, code, dépendances, IaC, images ; le modèle 14B consolide leurs résultats en une revue unique et priorisée, avec une suggestion de correction par vulnérabilité.

Démonstration — le code soumis

```
wire_transfer.php
1  <?php
2  // — Secrets codés en dur
3  $JWT_SIGNING_KEY = "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJyb2xlIjoiaWRTaW4ifQ.Hk9sL2pQ
4  $INTERNAL_API_TOKEN = "f4c7e2b9d6a1083f5a2c1e4b7d8f0a3c6e9b2d5f8a1c4e7b";
5  $DB_PASSWORD = "Pr0d_R00t_K9x2Lm7Qp4Vn8Tz3Wb6Hc1Df5Gj0";
6  // — SQL injection
7  function getAccountBalance($db) {
8      $sql = "SELECT balance FROM accounts WHERE id = " . $_GET['account_id'];
9      return mysqli_query($db, $sql);
10 }
11 // — Command injection —
12 function generateStatement() {
13     $cmd = "wkhtmltopdf /statements/" . $_GET['account_id'] . ".html /out.pdf";
14     return shell_exec($cmd);
15 }
16 // — SSRF
17 function fetchExchangeRate($currency) {
18     return file_get_contents($_GET['rate_provider'] . "?cur=" . $currency);
19 }
20 // — Désérialisation non sécurisée —
21 function loadTransferSession() {
22     return unserialize($_COOKIE['session']);
23 }
24 // — XSS réfléchi
25 function showConfirmation() {
26     echo "<div>Virement confirmé pour " . $_GET['recipient'] . "</div>";
27 }
28 // — Hachage faible (MD5)
29 function transactionRef($payload) {
30     return md5($payload . time());
31 }
32 // — Mot de passe transmis dans l'URL
33 function authorizeTransfer($amount) {
34     if ($_GET['admin_password'] === "admin123") {
35         return doTransfer($amount);
36     }
37     return false;
38 }
```

wire_transfer.php — sept vulnérabilités volontaires + secrets en dur

```
requirements.txt
1  Django==2.2.0
2  Flask==0.12.2
3  requests==2.19.1
4  PyYAML==5.1
5  Jinja2==2.10
6  urllib3==1.24.1
7
```

requirements.txt — dépendances obsolètes

```
Dockerfile
1  FROM python:3.8
2  ADD . /app
3  WORKDIR /app
4  EXPOSE 8000
5  CMD ["python3", "-m", "http.server", "8000"]
6
```

Dockerfile — conteneur en root

Une seule Pull Request

Trois fichiers : injection SQL, commande, SSRF, désérialisation, XSS, MD5, mot de passe en URL ; clé JWT, jeton et mot de passe en clair.

Démonstration — l'agent au travail

```

-4405-99f4-e366464f3fde
2026-06-28T14:00:26.941951Z [info] ] docker_build_success
2026-06-28T14:00:26.943745Z [info] ] scanning_dependencies
2026-06-28T14:00:26.967252Z [info] ] scanning_iac
2026-06-28T14:00:26.986061Z [info] ] scanning_sast
2026-06-28T14:00:27.006101Z [info] ] scanning_image
464f3fde
2026-06-28T14:00:27.027008Z [info] ] scanning_secrets
2026-06-28T14:00:27.049131Z [info] ] scanning_filesystem
2026-06-28T14:00:27.256704Z [info] ] trivy_parsed
05-99f4-e366464f3fde total=64
2026-06-28T14:00:27.257833Z [info] ] scan_complete
2026-06-28T14:00:27.849367Z [info] ] gitleaks_clean
2026-06-28T14:00:27.850180Z [info] ] scan_complete
2026-06-28T14:00:30.508343Z [info] ] osv_parsed
2026-06-28T14:00:30.510079Z [info] ] scan_complete
2026-06-28T14:00:33.515066Z [info] ] checkov_parsed
2026-06-28T14:00:33.516071Z [info] ] scan_complete
2026-06-28T14:00:33.769437Z [info] ] semgrep_parsed
2026-06-28T14:00:33.770251Z [info] ] scan_complete
2026-06-28T14:00:41.035359Z [info] ] trivy_parsed
2-8a78-4405-99f4-e366464f3fde total=6324
2026-06-28T14:00:41.037974Z [info] ] scan_complete
2026-06-28T14:00:41.047609Z [info] ] scan_result_saved
2026-06-28T14:00:41.050488Z [info] ] scan_result_saved
2026-06-28T14:00:41.052003Z [info] ] scan_result_saved
2026-06-28T14:00:41.789284Z [info] ] scan_full_complete
405-99f4-e366464f3fde
2026-06-28T14:00:41.795023Z [info] ] analyze_review_start
2026-06-28 14:01:07 [info] ] alertmanager_webhook_received count=1 service=webhooks
INFO: 172.20.0.10:37718 - "POST /webhooks/alertmanager HTTP/1.1" 200 OK
2026-06-28 14:01:07 [info] ] slack_notification_sent channel=C0ARJD4HJK5 service=slack_api
2026-06-28 14:01:07 [info] ] alerts_dispatched firing=0 resolved=1 service=webhooks
2026-06-28 14:10:07 [info] ] alertmanager_webhook_received count=1 service=webhooks
INFO: 172.20.0.10:35446 - "POST /webhooks/alertmanager HTTP/1.1" 200 OK
2026-06-28 14:10:07 [info] ] slack_notification_sent channel=C0ARJD4HJK5 service=slack_api
2026-06-28 14:10:07 [info] ] alerts_dispatched firing=1 resolved=0 service=webhooks
2026-06-28T14:19:43.589464Z [info] ] HTTP Request: POST http://ollama:11434/api/chat "HTTP/1.1 200 OK" task_id=45ca18c2-8a78-4405-99f4-e366464f3fde

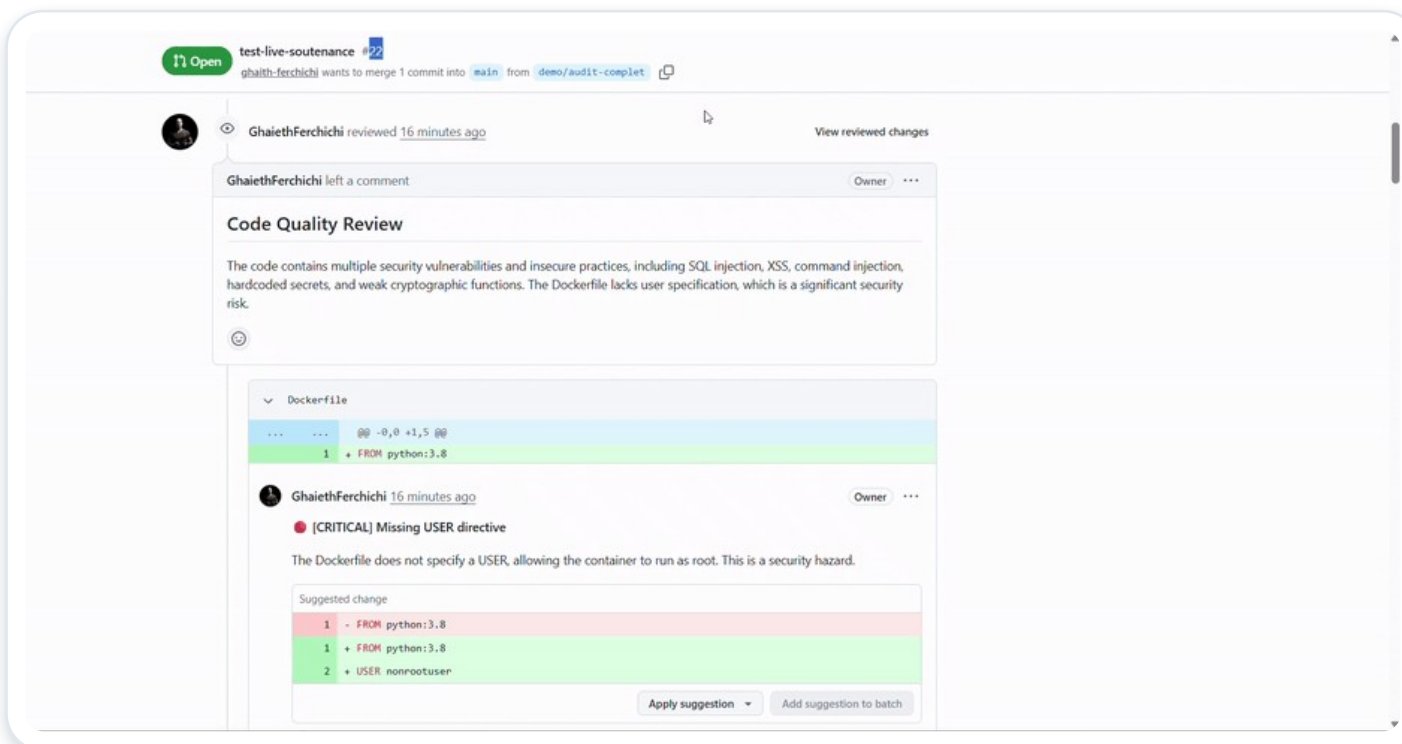
```

Sans intervention humaine

- Webhook vérifié (HMAC-SHA256)
- Classification par le modèle 7B
- Six scanners en parallèle
- Semgrep 43 · OSV 111 · Trivy 64
- Analyse approfondie par le 14B
- 100 % local — rien ne sort du VPS

Journaux de l'agent en direct — webhook → clone → classification → six scanners → analyse 14B

Démonstration — verdict et revue inline



Synthèse CRITIQUE, puis huit commentaires inline sur l'onglet « Files changed »

Huit commentaires inline

- Injection SQL · commande · SSRF
- Désérialisation · XSS · MD5
- Mot de passe transmis dans l'URL
- Dockerfile sans directive USER
- + une suggestion de correction par point

VERDICT : BLOCK



DevSecOps Agent APPLI 15 h 09

SECURITY AI AGENT — PR #22 review complete

Repo: GhaiethFerchichi/Vunl-application

Risk: CRITICAL | Verdict: BLOCK

Notification Slack — équipe prévenue

Bénéfices et résultats

Critère	Revue manuelle	Agent IA
Délai	jusqu'à 24 h (selon le relecteur)	15–25 min
Couverture	variable	OWASP Top 10 systématique
Gate CI/CD	aucun	APPROVE / CHANGES / BLOCK
Confidentialité	—	100 % local (BCT)

7 / 7

vulnérabilités plantées
détectées sur la PR #22
+ corpus de huit revues réelles

Supervision complète du VPS

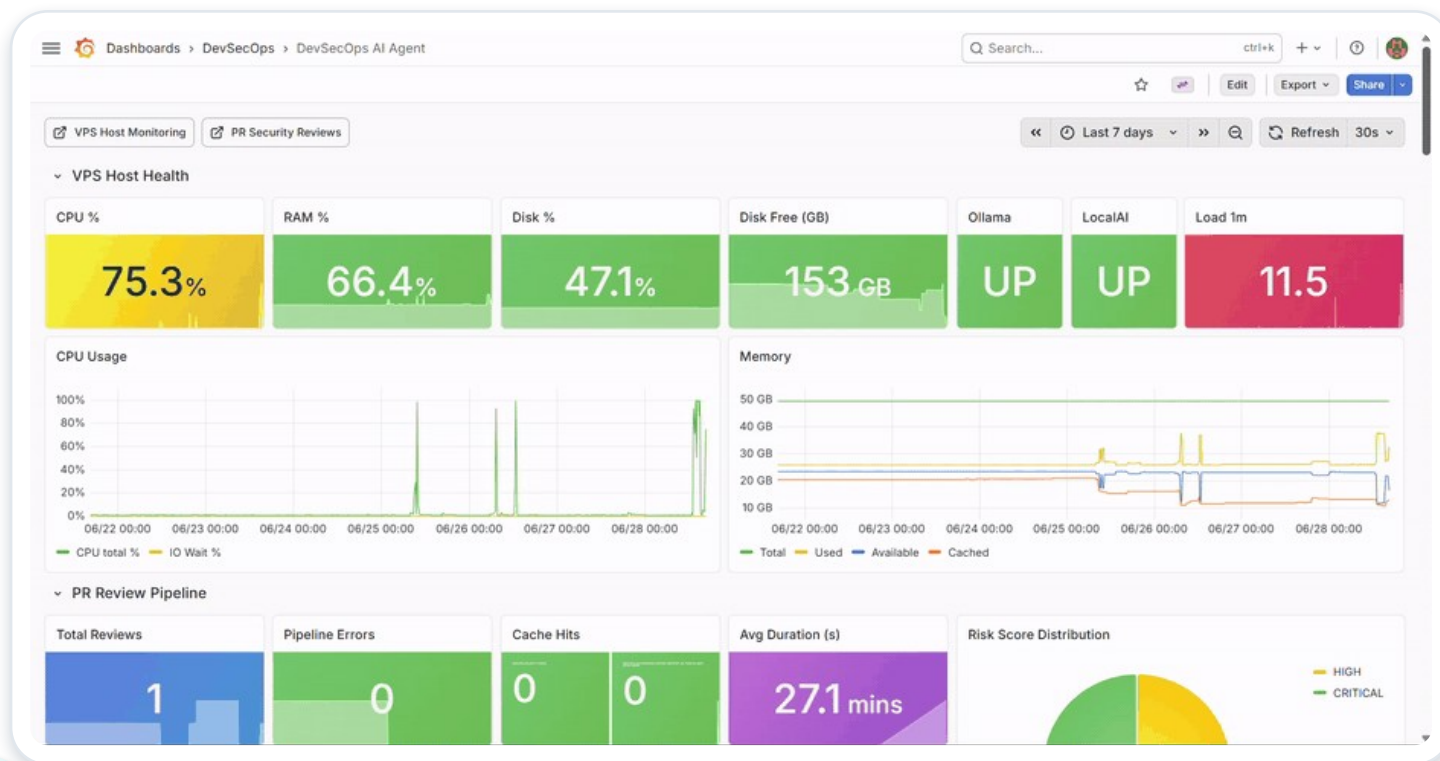


Tableau de bord DevSecOps en direct — rafraîchissement 30 s

Quatre piliers open source

Prometheus

collecte des métriques

VictoriaMetrics

stockage longue durée

Grafana

trois tableaux de bord

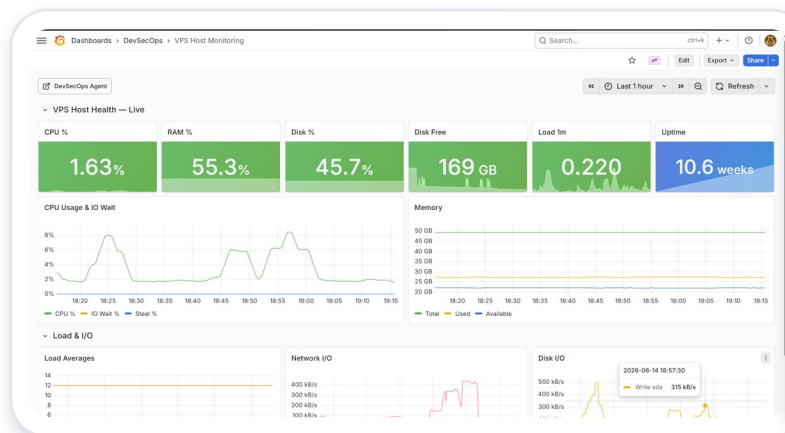
AlertManager

quinze règles d'alerte

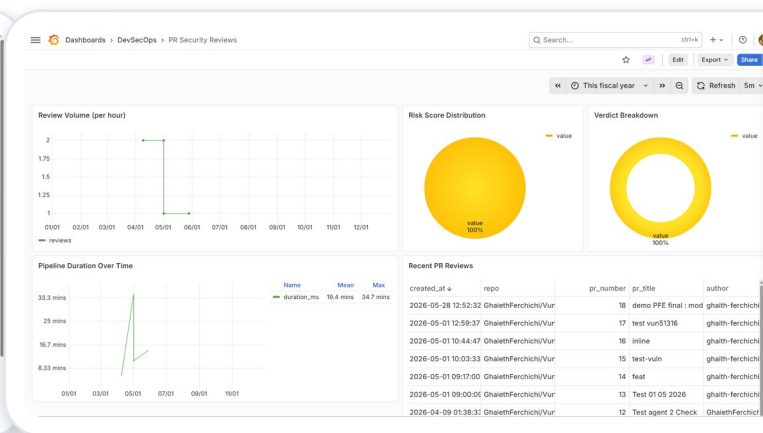
28 métriques • 15 alertes • 3 dashboards

Métriques conteneurs via le socket Docker, après l'échec de cAdvisor sur notre version ; les deux moteurs d'inférence comparés en continu.

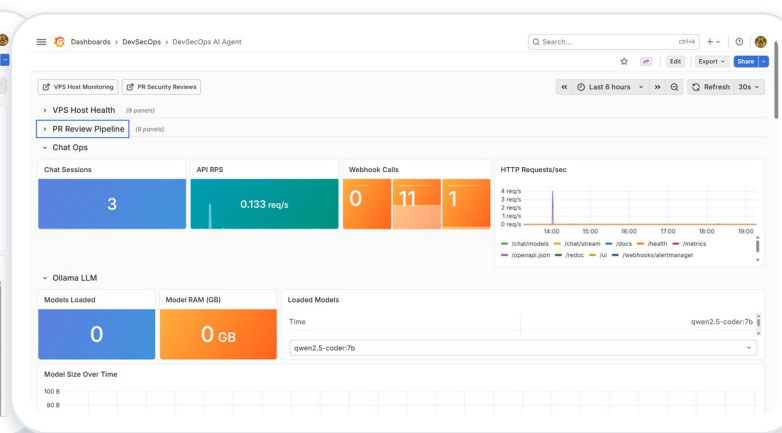
Une supervision opérationnelle, reliée à Slack



VPS — hôte



Revue de PR



Agent & LLM

The message from the DevSecOps Agent shows:

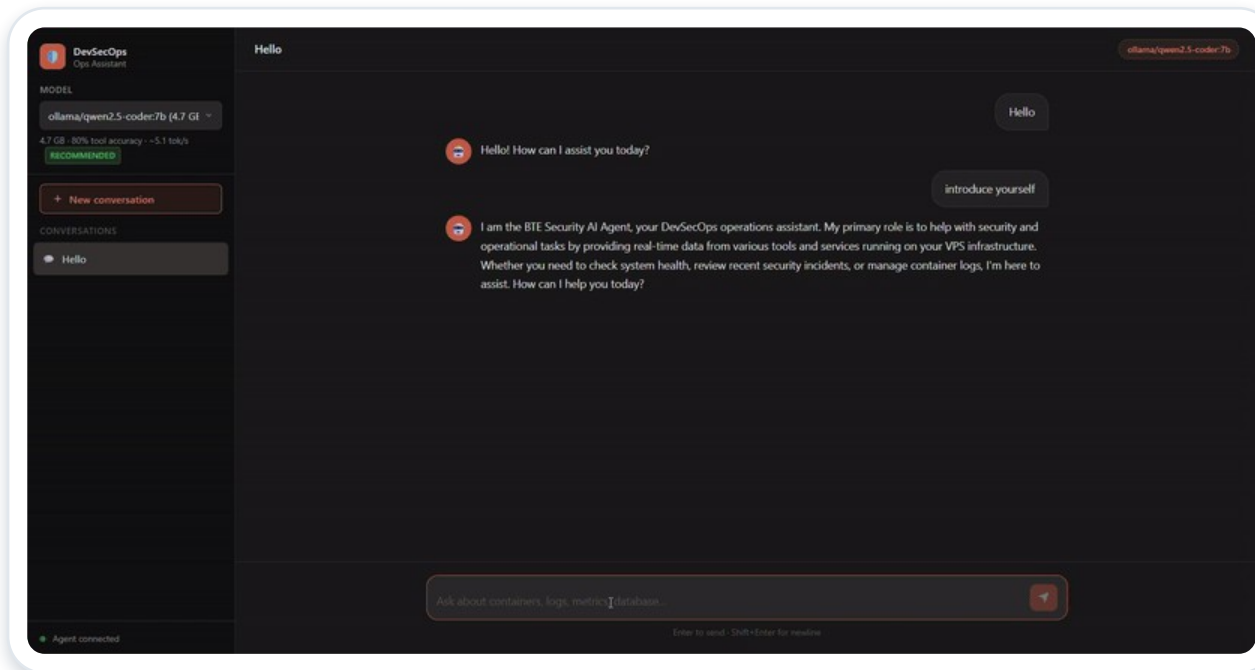
- 1 Alert(s) Firing — BTE VPS**
- HostHighCPU** (Severity: WARNING)
- Host CPU at 99.37% — may impact LLM inference speed.
- SECURITY AI AGENT — PR #18 review complete**
- Repo: GhaithFerchichi/Vunl-application
- Risk: HIGH | Verdict: BLOCK
- Resolved: HostHighCPU**

Canal #bte-security-ai-agent — alerte réelle, émise puis résolue

La supervision agit — elle ne décore pas

- Une alerte réelle se déclenche : CPU de l'hôte à 99 %, notifiée sur Slack
- L'agent confirme ensuite la résolution automatique de l'incident
- Le même canal reçoit le verdict de chaque revue de Pull Request

Assistant conversationnel et opérations autonomes



« Donne-moi la santé de mon VPS »

L'agent invoque un outil réel (**vps_status**) et répond avec la valeur mesurée — jamais inventée.

Et il s'auto-exploite :

- Scheduler autonome (tâches asyncio)
- Health digest Slack quotidien (09:00 UTC)
- Gardien disque auto-réparateur à 90 %

Tests et validation de la plateforme

✓ HMAC-SHA256

signature invalide rejetée en 403 ; signature valide acceptée en 202

✓ Déduplication Redis

deux webhooks identiques ne déclenchent qu'un seul pipeline (SET NX)

✓ Circuit breaker LLM

Ollama indisponible : repli sur la classification par extensions, sans blocage

✓ Gardien disque

au-delà de 90 % : nettoyage Docker automatique et notification Slack

Checklist de validation finale : quatorze vérifications de bout en bout, toutes validées avant la livraison.

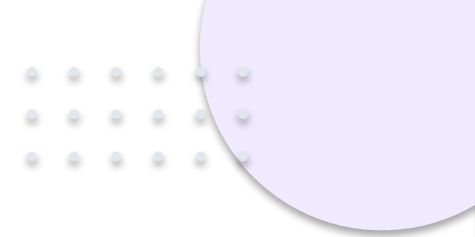
05

Conclusion

Limites, perspectives et bilan du projet



Limites assumées et perspectives



Limites assumées

- Inférence CPU : 15–25 min/PR — le prix de la confidentialité
- Corpus d'évaluation restreint (jeu contrôlé + 8 revues réelles)
- Diff = entrée non fiable → findings scanners déterministes, ancrage renforcé du verdict prévu (défense en profondeur)

Perspectives

- Migration GPU : 30–50 tok/s → 2–5 min/PR (~3000 €)
- Passage à l'échelle de tous les dépôts BTE
- Tableau de bord de risque pour le RSSI



CONCLUSION

Un agent IA agentique, déployé en production à la BTE,
qui sécurise chaque Pull Request **sans qu'aucun code ne quitte la banque.**

24 h → 15 min

délai

7 / 7

détection

100 % local

conformité BCT

Plateforme remise à la BTE · Merci de votre attention